

Core Language Working Group "tentatively ready" Issues for the February, 2016 (Jacksonville) meeting

Section references in this document reflect the section numbering of document [WG21 N4567](#).

212. Implicit instantiation is not described clearly enough

Section: 14.7.1 [temp.inst] **Status:** tentatively ready **Submitter:** Christophe de Dinechin **Date:** 7 Mar 2000

Three points have been raised where the wording in 14.7.1 [temp.inst] may not be sufficiently clear.

1. In paragraph 4, the statement is made that

A class template specialization is implicitly instantiated... if the completeness of the class type affects the semantics of the program...

It is not clear what it means for the "completeness... [to affect] the semantics." Consider the following example:

```
template<class T> struct A;
extern A<int> a;

void *foo() { return &a; }

template<class T> struct A
{
    #ifdef OPTION
        void *operator &() { return 0; }
    #endif
};
```

The question here is whether it is necessary for template class A to declare an operator & for the semantics of the program to be affected. If it does not do so, the meaning of &a will be the same whether the class is complete or not and thus arguably the semantics of the program are not affected.

Presumably what was intended is whether the presence or absence of certain member declarations in the template class might be relevant in determining the meaning of the program. A clearer statement may be desirable.

2. Paragraph 5 says,

If the overload resolution process can determine the correct function to call without instantiating a class template definition, it is unspecified whether that instantiation actually takes place.

The intent of this wording, as illustrated in the example in that paragraph, is to allow a "smart" implementation not to instantiate class templates if it can determine that such an instantiation will not affect the result of overload resolution, even though the algorithm described in clause 13 [over] requires that all the viable functions be enumerated, including functions that might be found as members of specializations.

Unfortunately, the looseness of the wording allowing this latitude for implementations makes it unclear what "the overload resolution process" is — is it the algorithm in 13 [over] or something else? — and what "the correct function" is.

3. According to paragraph 6,

If an implicit instantiation of a class template specialization is required and the template is declared but not defined, the program is ill-formed.

Here, it is not clear what conditions "require" an implicit instantiation. From the context, it would appear that the intent is

to refer to the conditions in paragraph 4 that cause a specialization to be instantiated.

This interpretation, however, leads to different treatment of template and non-template incomplete classes. For example, by this interpretation,

```
class A;
template <class T> struct TA;
extern A a;
extern TA<int> ta;

void f(A*);
void f(TA<int>*);

int main()
{
    f(&a);    // well-formed; undefined if A
              // has operator &() member
    f(&ta);   // ill-formed: cannot instantiate
}
```

A different approach would be to understand "required" in paragraph 6 to mean that a complete type is required in the expression. In this interpretation, if an incomplete type is acceptable in the context and the class template definition is not visible, the instantiation is not attempted and the program is well-formed.

The meaning of "required" in paragraph 6 must be clarified.

(See also issues 204 and 63.)

Notes on 10/01 meeting:

It was felt that item 1 is solved by addition of the word "might" in the resolution for issue 63; item 2 is not much of a problem; and item 3 could be solved by changing "required" to "required to be complete".

Proposed resolution (January, 2016):

1. Change 14.7.1 [temp.inst] paragraph 1 as follows, moving the note and example from paragraph 6 and breaking it into two paragraphs:

Unless a class template specialization has been explicitly instantiated (14.7.2 [temp.explicit]) or explicitly specialized (14.7.3 [temp.expl.spec]), the class template specialization is implicitly instantiated when the specialization is referenced in a context that requires a completely-defined object type or when the completeness of the class type affects the semantics of the program. **[Note: In particular, if the semantics of an expression depend on the member or base class lists of a class template specialization, the class template specialization is implicitly generated. For instance, deleting a pointer to class type depends on whether or not the class declares a destructor, and a conversion between pointers to class type depends on the inheritance relationship between the two classes involved. —end note]** **[Example:**

```
template<class T> class B { /* ... */ };
template<class T> class D : public B<T> { /* ... */ };
```

```
void f(void*);
void f(B<int>*);
```

```
void g(D<int>* p, D<char>* pp, D<double>* ppp) {
    f(p);           // instantiation of D<int> required: call f(B<int>*)
    B<char>* q = pp; // instantiation of D<char> required:
                    // convert D<char>* to B<char>*
    delete ppp;     // instantiation of D<double> required
}
```

—end example] If a class template has been declared, but not defined, at the point of instantiation (14.6.4.1 [temp.point]), the instantiation yields an incomplete class type (3.9 [basic.types]). **[Example:**

```
template<class T> class X;
```

```
X<char> ch;           // error: incomplete type X<char>
```

—end example] **[Note:** Within a template declaration, a local class (9.8 [class.local]) or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, *exception-specifications*, and non-static data member initializers, if any). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated

as part of the instantiation of the entity within which the local class or enumeration is declared. —end note]

The implicit instantiation of a class template specialization causes the implicit instantiation of the declarations, but not...

2. Delete 14.7.1 [temp.inst] paragraph 6, moving the note and example to paragraph 1 as shown above:

A class template specialization is implicitly instantiated if the class type is used in a context that requires a completely-defined object type or if the completeness of the class type might affect the semantics of the program. [Note: In particular, if the semantics of an expression depend on the member or base class lists of a class template specialization, the class template specialization is implicitly generated. For instance, deleting a pointer to class type depends on whether or not the class declares a destructor, and conversion between pointer to class types depends on the inheritance relationship between the two classes involved. —end note] [Example:

```
template<class T> class B { /* ... */ };
template<class T> class D : public B<T> { /* ... */ };

void f(void*);
void f(B<int>*);

void g(D<int>* p, D<char>* pp, D<double>* ppp) {
    f(p); // instantiation of D<int> required: call f(B<int>*)
    B<char>* q = pp; // instantiation of D<char> required:
                    // convert D<char>* to B<char>*
    delete ppp; // instantiation of D<double> required
}
```

—end example]

3. Change 14.7.1 [temp.inst] paragraph 7 as follows:

If the **function selected by** overload resolution **process (13.3 [over.match])** can **determine the correct function to call** **be determined** without instantiating a class template definition, it is unspecified whether that instantiation actually takes place. [Example:...

4. Delete 14.7.1 [temp.inst] paragraphs 8-9:

If an implicit instantiation of a class template specialization is required and the template is declared but not defined, the program is ill-formed. [Example:

```
template<class T> class X;

X<char> ch; // error: definition of X required

—end example]
```

The implicit instantiation of a class template does not cause any static data members of that class to be implicitly instantiated.

238. Precision and accuracy constraints on floating point

Section: 5 [expr] **Status:** tentatively ready **Submitter:** Christophe de Dinechin **Date:** 31 Jul 2000

It is not clear what constraints are placed on a floating point implementation by the wording of the Standard. For instance, is an implementation permitted to generate a "fused multiply-add" instruction if the result would be different from what would be obtained by performing the operations separately? To what extent does the "as-if" rule allow the kinds of optimizations (e.g., loop unrolling) performed by FORTRAN compilers?

Proposed resolution (September, 2015):

Change 3.9.1 [basic.fundamental] paragraph 8 as follows:

There are three *floating point* types: `float`, `double`, and `long double`. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. The value representation of floating-point types is implementation-defined. [Note: This

International Standard imposes no requirements on the accuracy of floating-point operations; see also 18.3.2 [limits]. —end note *Integral* and *floating* types are collectively called *arithmetic* types. Specializations of the standard **library** template `std::numeric_limits` (18.3 [support.limits]) shall specify the maximum and minimum values of each arithmetic type for an implementation.

242. Interpretation of old-style casts

Section: 5.4 [expr.cast] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 30 Aug 2000

The meaning of an old-style cast is described in terms of `const_cast`, `static_cast`, and `reinterpret_cast` in 5.4 [expr.cast] paragraph 5. Ignoring `const_cast` for the moment, it basically says that if the conversion performed by a given old-style cast is one of those performed by `static_cast`, the conversion is interpreted as if it were a `static_cast`; otherwise, it's interpreted as if it were a `reinterpret_cast`, if possible. The following example is given in illustration:

```
struct A {};  
struct I1 : A {};  
struct I2 : A {};  
struct D : I1, I2 {};  
A *foo( D *p ) {  
    return (A*)( p ); // ill-formed static_cast interpretation  
}
```

The obvious intent here is that a derived-to-base pointer conversion is one of the conversions that can be performed using `static_cast`, so `(A*)(p)` is equivalent to `static_cast<A*>(p)`, which is ill-formed because of the ambiguity.

Unfortunately, the description of `static_cast` in 5.2.9 [expr.static.cast] does NOT support this interpretation. The problem is in the way 5.2.9 [expr.static.cast] lists the kinds of casts that can be performed using `static_cast`. Rather than saying something like "All standard conversions can be performed using `static_cast`," it says

An expression *e* can be explicitly converted to a type τ using a `static_cast` of the form `static_cast< $\tau$ >(e)` if the declaration " τ *t*(e);" is well-formed, for some invented temporary variable *t*.

Given the declarations above, the hypothetical declaration

```
A* t(p);
```

is NOT well-formed, because of the ambiguity. Therefore the old-style cast `(A*)(p)` is NOT one of the conversions that can be performed using `static_cast`, and `(A*)(p)` is equivalent to `reinterpret_cast<A*>(p)`, which is well-formed under 5.2.10 [expr.reinterpret.cast] paragraph 7.

Other situations besides ambiguity which might raise similar questions include access violations, casting from virtual base to derived, and casting pointers-to-members when virtual inheritance is involved.

Proposed resolution (October, 2015):

1. Change 5.2.9 [expr.static.cast] paragraph 2 as follows:

An lvalue of type "cv1 B", where B is a class type, can be cast to type "reference to cv2 D", where D is a class derived (Clause 10 [class.derived]) from B, if a valid standard conversion from "pointer to D" to "pointer to B" exists (4.10 [conv.ptr]); cv2 is the same cv-qualification as, or greater cv-qualification than, cv1, and, if B is neither a virtual base class of D nor a base class of a virtual base class of D, or if no valid standard conversion from "pointer to D" to "pointer to B" exists (4.10 [conv.ptr]), the program is ill-formed. The result has type "cv2 D". An xvalue of type "cv1 B" may can be cast to type "rvalue reference to cv2 D" with the same constraints as for an lvalue of type "cv1 B". If the object of type "cv1 B" is actually a subobject of an object of type D, the result refers to the enclosing object of type D. Otherwise, the behavior is undefined. [Example:...

2. Change 5.2.9 [expr.static.cast] paragraph 4 as follows:

An expression *e* can be explicitly converted to a type τ using a `static_cast` of the form `static_cast< $\tau$ >(e)` if the declaration " τ *t*(e);" is well-formed, for some invented temporary variable *t* (8.5 [dcl.init]) there is an implicit conversion sequence (13.3.3.1 [over.best.ics]) from *e* to τ , or if overload resolution for a direct-initialization (8.5 [dcl.init]) of an object or reference of type τ from *e* would find at least one viable function (13.3.2 [over.match.viable]). The effect of such an explicit conversion is the same as performing the declaration and initialization

T t(e);

for some invented temporary variable t (8.5 [dcl.init]) and then using the temporary variable as the result of the conversion. **[Note: The conversion is ill-formed when attempting to convert an expression of class type to an inaccessible or ambiguous base class. —end note]** The expression e is used as a glvalue if and only if the initialization uses it as a glvalue.

3. Change 5.2.9 [expr.static.cast] paragraph 11 as follows:

A prvalue of type “pointer to cv1 B”, where B is a class type, can be converted to a prvalue of type “pointer to cv2 D”, where D is a class derived (Clause 10 [class.derived]) from B, if ~~a valid standard conversion from “pointer to D” to “pointer to B” exists (4.10 [conv.ptr])~~, cv2 is the same cv-qualification as, or greater cv-qualification than, cv1; ~~and. If B is neither a virtual base class of D nor or a base class of a virtual base class of D, or if no valid standard conversion from “pointer to D” to “pointer to B” exists (4.10 [conv.ptr]), the program is ill-formed.~~ The null pointer value (4.10 [conv.ptr]) is converted to the null pointer value of the destination type. If the prvalue of type “pointer to cv1 B” points to a B that is actually a subobject of an object of type D, the resulting pointer points to the enclosing object of type D. Otherwise, the behavior is undefined.

4. Change 5.2.9 [expr.static.cast] paragraph 12 as follows:

A prvalue of type “pointer to member of D of type cv1 T” can be converted to a prvalue of type “pointer to member of B” of type cv2 T”, where B is a base class (Clause 10 [class.derived]) of D, if ~~a valid standard conversion from “pointer to member of B of type T” to “pointer to member of D of type T” exists (4.11 [conv.mem])~~, and cv2 is the same cv-qualification as, or greater cv-qualification than, cv1.⁷⁰ **If no valid standard conversion from “pointer to member of B of type T” to “pointer to member of D of type T” exists (4.11 [conv.mem]), the program is ill-formed.** The null member pointer value (4.11 [conv.mem]) is converted to the null member pointer value of the destination type. If class B contains the original member, or is a base or derived class of the class containing the original member, the resulting pointer to member points to the original member. Otherwise, the behavior is undefined. *[Note: although class B need not contain the original member, the dynamic type of the object with which indirection through the pointer to member is performed must contain the original member; see 5.5 [expr.mptr.oper]. —end note]*

1284. Should the lifetime of an array be independent of that of its elements?

Section: 3.8 [basic.life] **Status:** tentatively ready **Submitter:** Gabriel Dos Reis **Date:** 2011-04-02

The note in 3.8 [basic.life] paragraph 2 reads,

[Note: The lifetime of an array object starts as soon as storage with proper size and alignment is obtained, and its lifetime ends when the storage which the array occupies is reused or released. 12.6.2 [class.base.init] describes the lifetime of base and member subobjects. —end note]

This wording reflects an earlier version of paragraph 1 that deferred the start of an object's lifetime only for initialization of objects of class type. The note simply emphasized the implication that that the lifetime of a POD type or an array began immediately, even if lifetime of an array's elements began later.

The decomposition of POD types removed the mention of PODs, leaving only the array types, and when the normative text was changed to include aggregates whose members have non-trivial initialization, the note was overlooked.

It is not clear whether it would be better to update the note to emphasize the distinction between aggregates with non-trivial initialization and those without or to delete it entirely.

A possible related normative change to consider is whether the specification of paragraph 1 is sufficiently clear with respect to multidimensional arrays. The current definition of “non-trivial initialization” is:

An object is said to have non-trivial initialization if it is of a class or aggregate type and it or one of its members is initialized by a constructor other than a trivial default constructor.

Presumably the top-level array of an N-dimensional array whose ultimate element type is a class type with non-trivial initialization would also have non-trivial initialization, but it's not clear that this wording says that.

A more radical change that came up in the discussion was whether the undefined behavior resulting from an lvalue-to-rvalue conversion of an uninitialized object in 4.1 [conv.lval] paragraph 1 would be better dealt with as a lifetime violation instead.

Proposed resolution (October, 2015):

Change 3.8 [basic.life] paragraphs 1 and 2 as follows:

The *lifetime* of an object is a runtime property of the object. An object is said to have *non-vacuous* initialization if it is of a class or aggregate type and it or one of its **members** **subobjects** is initialized by a constructor other than a trivial default constructor. [Note: initialization...

[Note: ~~The lifetime of an array object starts as soon as storage with proper size and alignment is obtained, and its lifetime ends when the storage which the array occupies is reused or released.~~ 12.6.2 [class.base.init] describes the lifetime of base and member subobjects. —end note]

1315. Restrictions on non-type template arguments in partial specializations

Section: 14.5.5 [temp.class.spec] **Status:** tentatively ready **Submitter:** Johannes Schaub **Date:** 2011-05-12

The rationale for the restriction in 14.5.5 [temp.class.spec] paragraph 8, first bullet is not clear:

- A partially specialized non-type argument expression shall not involve a template parameter of the partial specialization except when the argument expression is a simple *identifier*. [Example:

```
template <int I, int J> struct A {};
template <int I> struct A<I+5, I*2> {}; // error
template <int I, int J> struct B {};
template <int I> struct B<I, I> {};      // OK
```

—end example]

In the example, it's clear that `I` is non-deducible, but this rule prevents plausible uses like:

```
template <int I, int J> struct A {};
template <int I> struct A<I, I*2> {};
```

(See also issues 1647, 2033, and 2127.)

Proposed resolution (September, 2015):

Change 14.5.5 [temp.class.spec] bullet 8.1 as follows:

- ~~A partially specialized non-type argument expression shall not involve a template parameter of the partial specialization except when the argument expression is a simple *identifier*.~~ **Each *template-parameter* shall appear at least once in the *template-id* outside a non-deduced context.** [Example:

```
template <int I, int J> struct A {};
template <int I> struct A<I+5, I*2> {}; // error
```

```
template <int I, int J> struct B {};
template <int I> struct B A<I, I> {}; // OK
```

```
template <int I, int J, int K> struct B {};
template <int I> struct B<I, I*2, 2> {}; // OK
```

—end example]

1496. Triviality with deleted and missing default constructors

Section: 9.1 [class.name] **Status:** tentatively ready **Submitter:** Daniel Krügler **Date:** 2012-04-25

A default constructor that is defined as deleted is trivial, according to 12.1 [class.ctor] paragraph 5. This means that, according to 9 [class] paragraph 6, such a class can be trivial. If, however, the class has no default constructor because it has a user-declared constructor, the class is not trivial. Since both cases prevent default construction of the class, it is not clear why there is a difference in triviality between the cases.

(See also issue 1928.)

Notes from the October, 2012 meeting:

It was observed that this issue was related to issue 1344, as the current specification allows adding a default constructor by adding default arguments to the definition of a constructor. The resolution of that issue should also resolve this one.

Notes from the September, 2013 meeting:

It was decided to resolve issue 1344 separately from this issue, so this issue now requires its own resolution.

Proposed resolution (October, 2015) [SUPERSEDED]:

Change 9 [class] paragraph 6 as follows:

A *trivial class* is a class that ~~has a default constructor (12.1 [class.ctor]), has no non-trivial default constructors, and is~~ trivially copyable **and has one or more default constructors (12.1 [class.ctor]), all of which are either trivial or deleted and at least one of which is not deleted.** [Note: In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. —end note]

1638. Declaring an explicit specialization of a scoped enumeration

Section: 7.2 [dcl.enum] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2013-03-12

There is no syntax currently for declaring an explicit specialization of a member scoped enumeration. A declaration (not a definition) of such an explicit specialization most resembles an *opaque-enum-declaration*, but the grammar for that requires that the name be a simple identifier, which will not be the case for an explicit specialization of a member enumeration. This could be remedied by adding a *nested-name-specifier* to the grammar with a restriction that a *nested-name-specifier* only appear in an explicit specialization.

Proposed resolution (October, 2015):

1. Change the grammar in 7.2 [dcl.enum] paragraph 1 as follows:

opaque-enum-declaration:
enum-key attribute-specifier-seq_{opt} ***nested-name-specifier_{opt}*** *identifier enum-base_{opt}* ;

2. Add the following at the end of 7.2 [dcl.enum] paragraph 1:

If an *opaque-enum-declaration* contains a *nested-name-specifier*, the declaration shall be an explicit specialization (14.7.3 [temp.expl.spec]).

1872. Instantiations of constexpr templates that cannot appear in constant expressions

Section: 7.1.5 [dcl.constexpr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-02-17

According to 7.1.5 [dcl.constexpr] paragraph 6,

If the instantiated template specialization of a constexpr function template or member function of a class template would fail to satisfy the requirements for a constexpr function or constexpr constructor, that specialization is still a constexpr function or constexpr constructor, even though a call to such a function cannot appear in a constant expression.

The restriction on appearing in a constant expression assumes the previous wording that made such a specialization non-constexpr, and a call to a non-constexpr function cannot appear in a constant expression. With the current wording, however, there is no normative restriction against calls to such specializations. 5.20 [expr.const] should be updated to include such a prohibition.

Proposed resolution (January, 2016):

Add the following bullet following 5.20 [expr.const] bullet 2.3:

A *conditional-expression* *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (1.9 [intro.execution]), would evaluate one of the following expressions:

- ...
- an invocation of an undefined `constexpr` function or an undefined `constexpr` constructor;
- **an invocation of an instantiated `constexpr` function or `constexpr` constructor that fails to satisfy the requirements for a `constexpr` function or `constexpr` constructor (7.1.5 [dcl.constexpr]);**
- ...

1992. `new (std::nothrow) int[N]` can throw

Section: 5.3.4 [expr.new] **Status:** tentatively ready **Submitter:** Martin Sebor **Date:** 2014-08-27

According to 5.3.4 [expr.new] paragraph 7,

If the expression, after converting to `std::size_t`, is a core constant expression and the expression is erroneous, the program is ill-formed. Otherwise, a *new-expression* with an erroneous expression does not call an allocation function and terminates by throwing an exception of a type that would match a handler (15.3 [except.handle]) of type `std::bad_array_new_length` (18.6.2.2 [new.badlength]).

This wording makes no provision for an expression like

```
new (std::nothrow) int[N]
```

which most programmers would intuitively expect not to throw an exception under any condition.

Proposed resolution (May, 2015) [SUPERSEDED]:

Change the last part of 5.3.4 [expr.new] paragraph 7 as follows, converting the running text into bullets, and making the last sentence into a paragraph 8:

...If the expression, **is erroneous** after converting to `std::size_t`;

- **if the expression** is a core constant expression ~~and the expression is erroneous~~, the program is ill-formed;
- ~~Otherwise~~ **otherwise**, ~~a new-expression with an erroneous expression does not call~~ an allocation function **is not called; instead**
 - **if the allocation function that would have been called is non-throwing (15.4 [except.spec]), the value of the new-expression is the null pointer value of the required result type;**
 - ~~and~~ **otherwise, the new-expression** terminates by throwing an exception of a type that would match a handler (15.3 [except.handle]) of type `std::bad_array_new_length` (18.6.2.2 [new.badlength]).

When the value of the *expression* is zero, the allocation function is called to allocate an array with no elements.

Notes from the October, 2015 meeting:

The text in 15.4 paragraph 15 should also be changed.

Proposed resolution (January, 2016):

1. Change 5.3.4 [expr.new] paragraph 7 as follows, dividing the running text into bullets and making the last sentence into a new paragraph:

The expression in a *noptr-new-declarator* is erroneous if:

- ...

If the expression, **is erroneous** after converting to `std::size_t`;

- **if the expression** is a core constant expression ~~and the expression is erroneous~~, the program is ill-formed;
- ~~Otherwise~~ **otherwise**, ~~a new-expression with an erroneous expression does not call~~ an allocation function

and is not called; instead

- if the allocation function that would have been called has a non-throwing exception specification (15.4 [except.spec]), the value of the *new-expression* is the null pointer value of the required result type;
- otherwise, the *new-expression* terminates by throwing an exception of a type that would match a handler (15.3 [except.handle]) of type `std::bad_array_new_length` (18.6.2.2 [new.badlength]).

When the value of the expression is zero, the allocation function is called to allocate an array with no elements.

2. Change 15.4 [except.spec] paragraph 14 as follows:

The *set of potential exceptions of an expression* *e* is empty if *e* is a core constant expression (5.20 [expr.const]). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of *e*, including default argument expressions used in a function call, combined with a set *S* defined by the form of *e*, as follows:

- ...
- If *e* implicitly invokes a **one or more** functions (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if *e* is a full-expression (1.9 [intro.execution])), *S* is the **set of potential exceptions of the function union of:**
 - the sets of potential exceptions of all such functions, and
 - if *e* is a *new-expression* with a non-constant expression in the *nopt-new-declarator* (5.3.4 [expr.new]) and the allocation function selected for *e* has a non-empty set of potential exceptions, the set containing `std::bad_array_new_length`.
- ...
- If *e* is a *new-expression* with a non-constant expression in the *nopt-new-declarator* (5.3.4 [expr.new]), *S* consists of the type `std::bad_array_new_length`.

[Example:...

3. Change the example in 15.4 [except.spec] bullet 17.2 as follows:

```
struct A {
    A(int = (A(5), 0)) noexcept;
    A(const A&) throw();
    A(A&&) throw();
    ~A() throw(X);
};
struct B {
    B() throw();
    B(const B&) = default; // exception specification contains no types
    B(B&&, int = (throw Y(), 0)) noexcept;
    ~B() throw(Y);
};
int n = 7;
struct D : public A, public B {
    int * p = new (std::nothrow) int[n];
    // exception specification of D::D() contains X and std::bad_array_new_length
    // exception specification of D::D(const D&) contains no types
    // exception specification of D::D(D&&) contains Y
    // exception specification of D::~~D() contains X and Y
};
struct exp : std::bad_alloc {};
void *operator new[](size_t) throws(exp);
struct E : public A {
    int * p = new int[n];
    // exception specification of E::E() contains X, exp, and std::bad_array_new_length
};
```

According to 3.7 [basic.stc] paragraph 3,

The storage duration categories apply to references as well. The lifetime of a reference is its storage duration.

This is clearly not correct; references can have static storage duration but be dynamically initialized. Consider an example like:

```
extern int& r1;
int& f();
int& r2 = r1;    // #1
int& r1 = f();
int i = r2;      // #2
```

`r1` is not initialized until after its use at #1, so the initialization of `r2` should produce undefined behavior, as should the use of `r2` at #2.

The description of the lifetime of a reference should be deleted from 3.7 [basic.stc] and it should be described properly in 3.8 [basic.life].

Proposed resolution (September, 2015):

1. Change 3.7 [basic.stc] paragraph 3 as follows:

The storage duration categories apply to references as well. ~~The lifetime of a reference is its storage duration.~~

2. Change 3.8 [basic.life] paragraph 1 as follows:

The *lifetime* of an object **or reference** is a runtime property of the object **or reference**. An object is said to have...

3. Add the following as a new paragraph following 3.7 [basic.stc] paragraph 2:

[*Note:* The lifetime of an array object starts as soon as storage with proper size and alignment is obtained, and its lifetime ends when the storage which the array occupies is reused or released. 12.6.2 [class.base.init] describes the lifetime of base and member subobjects. —*end note*]

The lifetime of a reference begins when its initialization is complete. The lifetime of a reference ends as if it were a scalar object.

4. Change 3.8 [basic.life] paragraph 3 as follows:

The properties ascribed to objects **and references** throughout this International Standard apply for a given object **or reference** only during its lifetime. [*Note:*...

Change 5 [expr] paragraph 5 as follows:

If an expression initially has the type “reference to τ ” (8.3.2 [dcl.ref], 8.5.3 [dcl.init.ref]), the type is adjusted to τ prior to any further analysis. The expression designates the object or function denoted by the reference, and the expression is an lvalue or an xvalue, depending on the expression. **[*Note:* Before the lifetime of the reference has started or after it has ended, the behavior is undefined (see 3.8 [basic.life]). —*end note*]**

Drafting note: there is no change to 3.8 [basic.life] paragraph 4:

A program may end the lifetime of any object by reusing the storage which the object occupies or by explicitly calling the destructor for an object of a class type with a non-trivial destructor. For an object of a class type...

2032. Default *template-arguments* of variable templates

Section: 14.1 [temp.param] **Status:** tentatively ready **Submitter:** CWG **Date:** 2014-11-03

According to 14.1 [temp.param] paragraph 11,

If a *template-parameter* of a class template or alias template has a default *template-argument*, each subsequent *template-parameter* shall either have a default *template-argument* supplied or be a template parameter pack. If a *template-parameter* of a primary class template or alias template is a template parameter pack, it shall be the last *template-parameter*.

These requirements should apply to variable templates as well.

Proposed resolution (September, 2015):

Change 14.1 [temp.param] paragraph 11 as follows:

If a *template-parameter* of a class template, **variable template**, or alias template has a default *template-argument*, each subsequent *template-parameter* shall either have a default *template-argument* supplied or be a template parameter pack. If a *template-parameter* of a primary class template, **primary variable template**, or alias template is a template parameter pack, it shall be the last *template-parameter*. A template parameter pack...

2033. Redundant restriction on partial specialization argument

Section: 14.5.5 [temp.class.spec] **Status:** tentatively ready **Submitter:** CWG **Date:** 2014-11-04

Bullets 8.3 and 8.4 of 14.5.5 [temp.class.spec] say,

Within the argument list of a class template partial specialization, the following restrictions apply:

- ...
- The argument list of the specialization shall not be identical to the implicit argument list of the primary template.
- The specialization shall be more specialized than the primary template (14.5.5.2 [temp.class.order]).
- ...

The former is implied by the latter and should be omitted.

(See also issues 1315, 1647, and 2127.)

Proposed resolution (September, 2015):

Delete bullet 8.3 of 14.5.5 [temp.class.spec]:

- ~~The argument list of the specialization shall not be identical to the implicit argument list of the primary template.~~
-

2038. Document C++14 incompatibility of new braced deduction rule

Section: C.4 [diff.cpp14] **Status:** tentatively ready **Submitter:** Jonathan Caves **Date:** 2014-11-08

The adoption of document N3922 at the November, 2014 meeting introduces a new incompatibility that should be documented in Annex C [diff]:

```
int x1 = 1;
auto x2{x1};    // Is now int before was std::initializer<int>
```

Proposed resolution (September, 2015):

Insert the following as a new section following C.4.1 [diff.cpp14.lex]:

C.4.2 Clause 7: Declarations

[diff.cpp14.dcl]

7.1.6.4 [dcl.spec.auto]

Change: auto deduction from *braced-init-list*

Rationale: More intuitive deduction behavior

Effect on original feature: Valid C++14 code may fail to compile or may change meaning in this International Standard. For example:

```
auto x1{1};    // Was std::initializer_list<int>, now int
```

2039. Constant conversions to `bool`

Section: 15.4 [except.spec] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-11-09

According to 15.4 [except.spec] paragraph 1,

In a *noexcept-specification*, the *constant-expression*, if supplied, shall be a constant expression (5.20 [expr.const]) that is contextually converted to `bool` (Clause 4 [conv]).

This allows the expression to have any type that can be converted to `bool`, which is too lenient; it should instead say something like “a converted constant expression of type `bool` (5.20 [expr.const]).” This would include the conversion to `bool` in the determination of whether the expression is constant or not and would also disallow narrowing conversions.

A similar consideration applies to `static_assert` (7 [dcl.dcl] paragraph 6), which should probably be recast to something like, “an expression that is a constant expression (5.20 [expr.const]) after contextual conversion to `bool` (Clause 4 [conv]).”

Proposed resolution (September, 2015):

1. Change 5.20 [expr.const] paragraph 4 as follows:

...binds directly. [Note: such expressions may be used in new expressions (5.3.4 [expr.new]), as case expressions (6.4.2 [stmt.switch]), as enumerator initializers if the underlying type is fixed (7.2 [dcl.enum]), as array bounds (8.3.4 [dcl.array]), and as non-type template arguments (14.3 [temp.arg]). —end note] **A contextually converted constant expression of type `bool` is an expression, contextually converted to `bool` (Clause 4 [conv]), where the converted expression is a constant expression and the conversion sequence contains only the conversions above.**

2. Change 7 [dcl.dcl] paragraph 6 as follows:

In a *static_assert-declaration*, the *constant-expression* shall be a **contextually converted** constant expression **of type `bool`** (5.20 [expr.const]) ~~that can be contextually converted to `bool` (Clause 4 [conv])~~. If the value of the expression when so converted is true...

3. Change 15.4 [except.spec] paragraph 1 as follows:

In a *noexcept-specification*, the *constant-expression*, if supplied, shall be a **contextually converted** constant expression **of type `bool`** (5.20 [expr.const]) ~~that is contextually converted to `bool` (Clause 4 [conv])~~. A (token that follows *noexcept*...

2040. *trailing-return-type* no longer ambiguous

Section: 8 [dcl.decl] **Status:** tentatively ready **Submitter:** Jens Maurer **Date:** 2014-11-09

According to 8 [dcl.decl] paragraph 5,

The *type-id* in a *trailing-return-type* includes the longest possible sequence of *abstract-declarators*. [Note: This resolves the ambiguous binding of array and function declarators. [Example:

```
auto f()->int(*)[4]; // function returning a pointer to array[4] of int
                  // not function returning array[4] of pointer to int
```

—end example] —end note]

However, the grammar has changed since that rule and example were added; because *trailing-return-type* can only appear at the top level, there is no longer any potential ambiguity.

Proposed resolution (September, 2015):

Delete 8 [dcl.decl] paragraph 5:

The optional *attribute-specifier-seq* in a *trailing-return-type* appertains to the indicated return type. The *type-id* in a *trailing-return-type* includes the longest possible sequence of *abstract-declarators*. [Note: This resolves the ambiguous binding of array and function declarators. [Example:

```
auto f()->int(*)[4]; // function returning a pointer to array[4] of int
// not function returning array[4] of pointer to int
—end example] —end note]
```

2041. Namespace for explicit class template specialization

Section: 14.7.3 [temp.expl.spec] **Status:** tentatively ready **Submitter:** Jason Merrill **Date:** 2014-11-11

14.7.3 [temp.expl.spec] paragraph 2 says,

An explicit specialization shall be declared in a namespace enclosing the specialized template. An explicit specialization whose *declarator-id* is not qualified shall be declared in the nearest enclosing namespace of the template, or, if the namespace is inline (7.3.1 [namespace.def]), any namespace from its enclosing namespace set.

However, an explicit specialization of a class template does not have a *declarator-id*.

Proposed resolution (September, 2015):

Change 14.7.3 [temp.expl.spec] paragraph 2 as follows:

An explicit specialization shall be declared in a namespace enclosing the specialized template. An explicit specialization whose *declarator-id* **or class-head-name** is not qualified shall be declared in the nearest enclosing namespace of the template, or, if the namespace is inline (7.3.1 [namespace.def]), any namespace from its enclosing namespace set. Such a declaration...

2044. decltype(auto) and void

Section: 7.1.6.4 [dcl.spec.auto] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-11-14

The resolution of issue 1877 does not correctly handle `decltype(auto)` return types with `void` return expressions:

```
τ f();
decltype(auto) g() { return f(); }
```

fails when `τ` is `void`.

Suggested resolution:

Change 7.1.6.4 [dcl.spec.auto] paragraph 7 as follows:

...In the case of a return with no operand or with an operand of type `void`, the declared return type shall be `auto` **or** **decltype(auto)** and the deduced return type is `void`. Otherwise...

Proposed resolution (September, 2015):

Change 7.1.6.4 [dcl.spec.auto] paragraph 7 as follows:

When a variable declared using a placeholder type is initialized, or a `return` statement occurs in a function declared with a return type that contains a placeholder type, the deduced return type or variable type is determined from the type of its initializer. In the case of a return with no operand or with an operand of type `void`:

- **if the declared return type is `decltype(auto)`, then the deduced return type is `void`;**
- **otherwise,** the declared return type shall be **cv** `auto` and the deduced return type is **cv** `void`.

Otherwise, let `τ` be...

2047. Coordinating “throws anything” specifications

Section: 15.4 [except.spec] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-11-18

The resolutions of issues 330 and 1351 use different terminology for an exception specification that can throw anything: the former refers to a “(conceptual) set of all types,” while the latter uses a “pseudo-type, denoted by ‘any’.” These should be unified.

Proposed resolution (October, 2015) [SUPERSEDED]:

1. Change 15.4 [except.spec] paragraph 13 as follows:

A The set of potential exceptions of a given context is **either a type or a set of types** that might be thrown as an exception or a pseudo-type, denoted by “any”, that represents the situation where an exception of an arbitrary type might be thrown; **the (conceptual) set of all types is used to denote that an exception of arbitrary type might be thrown.** A subexpression *e*₁ of an expression *e* is an *immediate subexpression* if there is no subexpression *e*₂ of *e* such that *e*₁ is a subexpression of *e*₂.

2. Delete 15.4 [except.spec] paragraph 14:

~~The set of potential exceptions of a function, function pointer, or member function pointer *f* is defined as follows:~~

- ~~◦ If the exception specification of *f* is the set of all types, the set consists of the pseudo-type “any”.~~
- ~~◦ Otherwise, the set consists of every type in the exception specification of *f*.~~

3. Change 15.4 [except.spec] paragraph 15 as follows:

The *set of potential exceptions of an expression e* is empty if *e* is a core constant expression (5.20 [expr.const]). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of *e*, including default argument expressions used in a function call, combined with a set *S* defined by the form of *e*, as follows:

- If *e* is a function call (5.2.2 [expr.call]):
 - If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1 [expr.prim.general]), class member access (5.2.5 [expr.ref]), or pointer-to-member operation (5.5 [expr.mptr.oper]) whose *cast-expression* is an *id-expression*, *S* is the set of **potential exceptions types in the exception specification** of the entity selected by the contained *id-expression* (after overload resolution, if applicable).
 - Otherwise, *S* **contains the pseudo-type “any” is the set of all types.**
- If *e* implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if *e* is a full-expression (1.9 [intro.execution])), *S* is the set of **potential exceptions types in the exception specification** of the function.
- if *e* is a *throw-expression* (5.17 [expr.throw]), *S* consists of the type of the exception object that would be initialized by the operand, if present, or **is the pseudo-type “any” set of all types** otherwise.
- if *e* is a *dynamic_cast* expression that casts to a reference type and requires a run-time check (5.2.7 [expr.dynamic.cast]), *S* consists of the type `std::bad_cast`.
- if *e* is a *typeid* expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8 [expr.typeid]), *S* consists of the type `std::bad_typeid`.
- if *e* is a *new-expression* with a non-constant expression in the *nopttr-new-declarator* (5.3.4 [expr.new]), *S* consists of the type `std::bad_array_new_length`.

[Example: Given the following declarations

```
void f() throw(int);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw (double); };
```

the set of potential exceptions for some sample expressions is:

- for `f()`, the set consists of `int`;
- for `g()`, the set ~~consists of “any”~~ **is the set of all types**;
- for `new A`, the set ~~consists of “any”~~ **is the set of all types**;
- for `B()`, the set is empty;
- for `new D`, the set ~~consists of “any” and `double`~~ **is the set of all types**. **[Note: This set conceptually includes the type `double`. —end note]**

—end example]

4. Change 15.4 [except.spec] paragraph 16 as follows:

Given a `A` member function `f` of some class `x`, where `f` is an inheriting constructor (N4527_12.9 [class.inhctor]) or an implicitly-declared special member function, ~~the set of potential exceptions of the implicitly-declared member function `f`~~ **is considered to have an implicit exception specification that consists of all the members from the following sets...**

5. Delete the normative portion of 15.4 [except.spec] paragraph 17 and merge the note and example into the preceding paragraph, as follows:

~~An inheriting constructor (N4527_12.9 [class.inhctor]) and an implicitly-declared special member function (Clause 12 [special]) are considered to have an implicit exception specification, as follows, where `S` is the set of potential exceptions of the implicitly-declared member function:~~

- ~~if `S` contains the pseudo-type “any”, the implicit exception specification is the set of all types;~~
- ~~otherwise, the implicit exception specification contains all the types in `S`.~~

[Note: An instantiation of an inheriting constructor template...

Additional note (November, 2015):

The base text underlying the preceding proposed resolution was changed at the October, 2015 meeting by the adoption of paper P0136R1. As a result, this issue has been returned to "drafting" status to allow reconciliation of the two sets of changes.

Proposed resolution (January, 2016):

1. Change 15.4 [except.spec] paragraph 12 as follows:

A ~~The set of~~ potential exceptions of a given context is ~~either a type~~ **a set of types** that might be thrown as an exception ~~or a pseudo-type, denoted by “any”, that represents the situation where an exception of an arbitrary type might be thrown;~~ **the (conceptual) set of all types is used to denote that an exception of arbitrary type might be thrown**. A subexpression `e1` of an expression `e` is an *immediate subexpression* if there is no subexpression `e2` of `e` such that `e1` is a subexpression of `e2`.

2. Delete 15.4 [except.spec] paragraph 13:

~~The set of potential exceptions of a function, function pointer, or member function pointer `f` is defined as follows:~~

- ~~If the exception specification of `f` is the set of all types, the set consists of the pseudo-type “any”;~~
- ~~Otherwise, the set consists of every type in the exception specification of `f`.~~

3. Change 15.4 [except.spec] paragraph 14 as follows:

The *set of potential exceptions of an expression `e`* is empty if `e` is a core constant expression (5.20 [expr.const]). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of `e`, including default argument expressions used in a function call, combined with a set `S` defined by the form of `e`, as follows:

- If `e` is a function call (5.2.2 [expr.call]):
 - If its *postfix-expression* is a (possibly parenthesized) *id-expression* (5.1.1 [expr.prim.general]), class member access (5.2.5 [expr.ref]), or pointer-to-member operation (5.5 [expr.mptr.oper]) whose *cast-expression* is an *id-expression*, `S` is the set of ~~potential exceptions~~ **types in the exception specification** of the entity selected by the contained *id-expression* (after overload resolution, if applicable).

- Otherwise, if the *postfix-expression* has type “noexcept function” or “pointer to noexcept function”, *S* is the empty set.
 - Otherwise, *S* contains the pseudo-type “any” is the set of all types.
- If *e* implicitly invokes a function (such as an overloaded operator, an allocation function in a *new-expression*, or a destructor if *e* is a full-expression (1.9 [intro.execution])), *S* is the set of potential exceptions types in the exception specification of the function.
 - If *e* initializes an object of type *D* using an inherited constructor for a class of type *B* (12.6.3 [class.inhctor.init]), *S* also contains the sets of potential exceptions of the implied constructor invocations for subobjects of *D* that are not subobjects of *B* (including default argument expressions used in such invocations) as selected by overload resolution, and the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* (12.6.2 [class.base.init]).
 - If *e* is a *throw-expression* (5.17 [expr.throw]), *S* consists of the type of the exception object that would be initialized by the operand, if present, or is the pseudo-type “any” set of all types otherwise.
 - If *e* is a *dynamic_cast* expression that casts to a reference type and requires a run-time check (5.2.7 [expr.dynamic.cast]), *S* consists of the type `std::bad_cast`.
 - If *e* is a *typeid* expression applied to a glvalue expression whose type is a polymorphic class type (5.2.8 [expr.typeid]), *S* consists of the type `std::bad_typeid`.
 - If *e* is a *new-expression* with a non-constant expression in the *noPtr-new-declarator* (5.3.4 [expr.new]), *S* consists of the type `std::bad_array_new_length`.

[Example: Given the following declarations

```
void f() throw(int);
void g();
struct A { A(); };
struct B { B() noexcept; };
struct D { D() throw (double); };
```

the set of potential exceptions for some sample expressions is:

- for *f()*, the set consists of `int`;
- for *g()*, the set consists of “any” is the set of all types;
- for *new A*, the set consists of “any” is the set of all types;
- for *B()*, the set is empty;
- for *new D*, the set consists of “any” and `double` is the set of all types.

—end example]

4. Change 15.4 [except.spec] paragraph 16 as follows:

Given an implicitly-declared special member function *f* of some class *X*, the set of potential exceptions of the implicitly-declared special member function *f* is considered to have an implicit exception specification that consists of all the members from the following sets:...

5. Delete the normative text of 15.4 [except.spec] paragraph 17 and merge the example with the preceding paragraph:

An implicitly-declared special member function (Clause 12 [special]) is considered to have an implicit exception specification, as follows, where *S* is the set of potential exceptions of the implicitly-declared special member function:

- if *S* contains the pseudo-type “any”, the implicit exception specification is the set of all types;
- otherwise, the implicit exception specification contains all the types in *S*.

[Example:...

2061. Inline namespace after simplifications

Section: 7.3.1 [namespace.def] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-12-18

After the resolution of issue 1795, 7.3.1 [namespace.def] paragraph 3 now says:

In a *named-namespace-definition*, the *identifier* is the name of the namespace. If the *identifier*, when looked up (3.4.1 [basic.lookup.unqual]), refers to a *namespace-name* (but not a *namespace-alias*) introduced in the declarative region in which the *named-namespace-definition* appears, the *namespace-definition extends* the previously-declared namespace. Otherwise, the *identifier* is introduced as a *namespace-name* into the declarative region in which the *named-namespace-definition* appears.

This appears to break code like the following:

```
namespace A {
    inline namespace b {
        namespace C {
            template<typename T> void f();
        }
    }
}

namespace A {
    namespace C {
        template<> void f<int>() { }
    }
}
```

because (by definition of “declarative region”) `c` cannot be used as an unqualified name to refer to `A::b::C` within `A` if its declarative region is `A::b`.

Proposed resolution (September, 2015):

Change 7.3.1 [namespace.def] paragraph 3 as follows:

In a *named-namespace-definition*, the *identifier* is the name of the namespace. If the *identifier*, when looked up (3.4.1 [basic.lookup.unqual]), refers to a *namespace-name* (but not a *namespace-alias*) **that was introduced in the declarative region namespace** in which the *named-namespace-definition* appears **or that was introduced in a member of the inline namespace set of that namespace**, the *namespace-definition extends* the previously-declared namespace. Otherwise, the *identifier* is introduced as a *namespace-name* into the declarative region in which the *named-namespace-definition* appears.

2063. Type/nontype hiding in class scope

Section: 3.3.1 [basic.scope.declarative] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2014-12-20

The type/nontype hiding rules (“`struct stat hack`”) do not apply in class scope. This is a C compatibility issue:

```
struct A {
    struct B { int x; } b;
    int B;    // Permitted in C
};
```

Since the type/nontype hiding rules exist for C compatibility, should this example be supported?

Proposed resolution (September, 2015):

Change 3.3.1 [basic.scope.declarative] paragraph 4 as follows:

Given a set of declarations in a single declarative region, each of which specifies the same unqualified name,

- they shall all refer to the same entity, or all refer to functions and function templates; or
- exactly one declaration shall declare a class name or enumeration name that is not a typedef name and the other declarations shall all refer to the same variable, **non-static data member**, or enumerator, or all refer to functions and function templates; in this case the class name or enumeration name is hidden (3.3.10 [basic.scope.hiding]). [Note: A namespace name or a class template name must be unique in its declarative

2064. Conflicting specifications for dependent *decltype-specifiers*

Section: 14.4 [temp.type] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-12-27

According to 14.6.2.1 [temp.dep.type] paragraph 9, a type is dependent if it is

denoted by `decltype(expression)`, where *expression* is type-dependent (14.6.2.2 [temp.dep.expr]).

However, 14.4 [temp.type] paragraph 2 says,

If an expression *e* involves a template parameter, `decltype(e)` denotes a unique dependent type. Two such *decltype-specifiers* refer to the same type only if their expressions are equivalent (14.5.6.1 [temp.over.link]). [Note: however, it may be aliased, e.g., by a *typedef-name*. —end note]

These seem to be in need of reconciliation.

Proposed resolution (January, 2016):

Change 14.4 [temp.type] paragraph 2 as follows:

If an expression *e* involves a template parameter is type-dependent (14.6.2.2 [temp.dep.expr]), `decltype(e)` denotes a unique dependent type. Two such *decltype-specifiers* refer to the same type only if their expressions are equivalent (14.5.6.1 [temp.over.link]). [Note: however, it such a type may be aliased, e.g., by a *typedef-name*. —end note]

2066. Does type-dependent imply value-dependent?

Section: 14.6.2.3 [temp.dep.constexpr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-01-09

Consider the following example:

```
template<int> struct X { typedef int type; };
template<typename T> struct Y {
    void f() { X<false ? this - this : 0>::type x; } // missing typename?
};
void g() { Y<void>().f(); }
```

This appears to be valid because the template argument expression is not value-dependent.

Until I discovered this, I had been assuming that any type-dependent expression is also value-dependent. The only exception to that appears to be the expression `this`, which may be type-dependent but is never value-dependent.

Now, `this` need not ever be value-dependent, because evaluation of it will never succeed when it appears as a subexpression of an expression that we're checking for constant-expression-ness. But if that's really what we want here, then the same applies to function parameters with dependent types, and probably a few other cases.

Proposed resolution (September, 2015):

Change 14.6.2.3 [temp.dep.constexpr] paragraph 4 as follows:

Expressions of the following form are value-dependent:

```
sizeof ... ( identifier )
this
fold-expression
```

2068. When can/must a defaulted virtual destructor be defined?

Section: 12.4 [class.dtor] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-01-12

The rules in 3.2 [basic.def.odr] and 12.4 [class.dtor] do not specify when the destructor for `B` can/must be defined:

```
struct A { virtual ~A(); };
struct B : A {};
int main() {
    A *p = new B;
    delete p;
}
```

An implementation should be allowed, but not required, to implicitly define a virtual special member function at any point where it has been declared, as well as being required to define it as described in 12.4 [class.dtor] paragraph 6, etc.

Proposed resolution (September, 2015):

Change 12.4 [class.dtor] paragraph 6 as follows:

A destructor that is defaulted and not defined as deleted is implicitly defined when it is odr-used (3.2 [basic.def.odr]) ~~to destroy an object of its class type (3.7 [basic.stc])~~ or when it is explicitly defaulted after its first declaration.

2069. Do destructors have names?

Section: 12.4 [class.dtor] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-01-13

According to 7.3.3 [namespace.udecl] paragraph 4,

[*Note:* Since destructors do not have names, a *using-declaration* cannot refer to a destructor for a base class...

However, 12.4 [class.dtor] paragraph 13 says,

In an explicit destructor call, the destructor name appears as a `~` followed by a *type-name* or *decltype-specifier* that denotes the destructor's class type...

See also 3.4.3.1 [class.qual] bullet 1.1:

a destructor name is looked up as specified in 3.4.3 [basic.lookup.qual];

Proposed resolution (September, 2015):

1. Change 3.4.3.1 [class.qual] bullet 1.1 as follows:

- ~~a destructor name is looked up~~ **the lookup for a destructor is** as specified in 3.4.3 [basic.lookup.qual];

2. Change 12.4 [class.dtor] paragraph 13 as follows:

In an explicit destructor call, the destructor ~~name appears as~~ **is specified by** a `~` followed by a *type-name* or *decltype-specifier* that denotes the destructor's class type. The invocation...

2071. typedef with no declarator

Section: 7.1.3 [dcl.typedef] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2014-01-16

There should be a rule to prohibit the almost certainly erroneous declaration

```
typedef struct X { };    // Missing declarator
```

Proposed resolution (September, 2015):

Change 7.1.3 [dcl.typedef] paragraph 1 as follows:

Declarations containing the *decl-specifier* `typedef` declare identifiers that can be used later for naming fundamental (3.9.1 [basic.fundamental]) or compound (3.9.2 [basic.compound]) types. The `typedef` specifier shall not be combined in a *decl-specifier-seq* with any other kind of specifier except a *type-specifier*, and it shall not be used in the *decl-*

specifier-seq of a *parameter-declaration* (8.3.5 [dcl.fct]) nor in the *decl-specifier-seq* of a *function-definition* (8.4 [dcl.fct.def]). **If a *typedef* specifier appears in a declaration without a *declarator*, the program is ill-formed.**

2079. `[]` appearing in a *balanced-token-seq*

Section: 7.6.1 [dcl.attr.grammar] **Status:** tentatively ready **Submitter:** Jonathan Caves **Date:** 2015-02-03

According to 7.6.1 [dcl.attr.grammar] paragraph 6,

Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier*. [Note: If two consecutive left square brackets appear where an *attribute-specifier* is not allowed, the program is ill-formed even if the brackets match an alternative grammar production. —end note]

In order to allow program fragments to appear within attributes, this restriction should not apply within the *balanced-token-seq* of an attribute.

Proposed resolution (September, 2015):

Change 7.6.1 [dcl.attr.grammar] paragraph 6 as follows:

Two consecutive left square bracket tokens shall appear only when introducing an *attribute-specifier* **or within the *balanced-token-seq* of an *attribute-argument-clause***. [Note: If two consecutive left square brackets appear where an *attribute-specifier* is not allowed, the program is ill-formed even if the brackets match an alternative grammar production. —end note] [Example:

```
int p[10];
void f() {
    int x = 42, y[5];
    int(p[[x] { return x; }()]); // error: invalid attribute on a nested
                                // declarator-id and not a function-style cast of
                                // an element of p.
    y[[ { return 2; }()] = 2;    // error even though attributes are not allowed
                                // in this context.
```

```
int i [[vendor::attr([[]])]]; // well-formed implementation-defined attribute.
}
```

—end example]

2085. Invalid example of adding special member function via default argument

Section: 3.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-02-13

The example in 3.2 [basic.def.odr] bullet 6.6 reads,

```
//translation unit 1:
struct X {
    X(int);
    X(int, int);
};
X::X(int = 0) { }
class D: public X { };
D d2;      // X(int) called by D()

//translation unit 2:
struct X {
    X(int);
    X(int, int);
};
X::X(int = 0, int = 0) { }
class D: public X { }; // X(int, int) called by D();
                       // D()'s implicit definition
```


// violates the ODR

Creating a special member function via default arguments added in an out-of-class definition, as is done here, is no longer permitted, so at a minimum the example should be removed. It is not clear whether there remain any cases to which the normative wording of bullet 6.6 would apply:

- if `D` is a class with an implicitly-declared constructor (12.1 [class.ctor]), it is as if the constructor was implicitly defined in every translation unit where it is odr-used, and the implicit definition in every translation unit shall call the same constructor for a base class or a class member of `D`.

If not, the entire bullet should be removed.

Proposed resolution (September, 2015):

Change 3.2 [basic.def.odr] bullet 6.6 as follows:

- if `D` is a class with an implicitly-declared constructor (12.1 [class.ctor]), it is as if the constructor was implicitly defined in every translation unit where it is odr-used, and the implicit definition in every translation unit shall call the same constructor for a ~~base class or a class member~~ **subobject** of `D`. [Example:

```
//translation unit 1:
struct X {
    X(int, int);
    X(int, int, int);
};
X::X(int, int = 0) { }
class D: public X {
    X x = 0;
};
D d2;           // X(int, int) called by D()

//translation unit 2:
struct X {
    X(int, int);
    X(int, int, int);
};
X::X(int, int = 0, int = 0) { }
class D: public X {
    X x = 0;
};
           // X(int, int, int) called by D();
           // D()'s implicit definition
           // violates the ODR
```

—end example]

2095. Capturing rvalue references to functions by copy

Section: 5.1.2 [expr.prim.lambda] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-03-07

According to 5.1.2 [expr.prim.lambda] paragraph 15,

An entity is captured by copy if it is implicitly captured and the *capture-default* is = or if it is explicitly captured with a capture that is not of the form `& identifier` or `& identifier initializer`. For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The declaration order of these members is unspecified. The type of such a data member is the type of the corresponding captured entity if the entity is not a reference to an object, or the referenced type otherwise.

It's not clear how to handle capture by copy when the entity is an rvalue reference to function. In particular, this appears to be a contradiction with 5.1.2 [expr.prim.lambda] paragraph 3,

An implementation shall not add members of rvalue reference type to the closure type.

Proposed resolution (September, 2015):

Change 5.1.2 [expr.prim.lambda] paragraph 15 as follows:

An entity is *captured by copy* if it is implicitly captured and the *capture-default* is = or if it is explicitly captured with a capture that is not of the form & *identifier* or & *identifier initializer*. For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The declaration order of these members is unspecified. The type of such a data member is **the referenced type if the entity is a reference to an object, an lvalue reference to the referenced function type if the entity is a reference to a function, or** the type of the corresponding captured entity **if the entity is not a reference to an object, or the referenced type** otherwise. ~~[Note: If the captured entity is a reference to a function, the corresponding data member is also a reference to a function. —end note]~~ A member of an anonymous union shall not be captured by copy.

2096. Constraints on literal unions

Section: 3.9 [basic.types] **Status:** tentatively ready **Submitter:** Agustín K-ballo Bergé **Date:** 2015-03-11

According to 3.9 [basic.types] bullet 10.5.3, all the members of a class type must be of non-volatile literal types. This seems overly constraining for unions; it would seem to be sufficient if at least one of its non-static members were of a literal type.

Proposed resolution (September, 2015):

Change 3.9 [basic.types] bullet 10.5 as follows:

A type is a *literal type* if it is:

- ...
- a possibly cv-qualified class type (Clause 9 [class]) that has all of the following properties:
 - it has a trivial destructor,
 - it is an aggregate type (8.5.1 [dcl.init.aggr]) or has at least one constexpr constructor or constructor template that is not a copy or move constructor, **and**
 - **if it is a union, at least one of its non-static data members is of non-volatile literal type, and**
 - **if it is not a union,** all of its non-static data members and base classes are of non-volatile literal types.

2098. Is `uncaught_exceptions()` per-thread?

Section: 15.5.3 [except.uncaught] **Status:** tentatively ready **Submitter:** Ville Voutilainen **Date:** 2015-03-14

The current specification of `std::uncaught_exceptions()` (15.5.3 [except.uncaught] paragraph 1) does not, but should, state that it is the number of uncaught exceptions in the current thread.

Proposed resolution (September, 2015):

Change 15.5.3 [except.uncaught] paragraph 1 as follows:

...The function `std::uncaught_exceptions()` (18.8.4 [uncaught.exceptions]) returns the number of uncaught exceptions **in the current thread.**

2104. Internal-linkage `constexpr` references and ODR requirements

Section: 3.2 [basic.def.odr] **Status:** tentatively ready **Submitter:** James Widman **Date:** 2015-03-17

In an example like:

```
extern int i;
namespace {
    constexpr int& r = i;
```

```

}
inline int f() { return r; }

```

use of `f()` in multiple translation units results in an ODR violation because of use of the internal-linkage reference `r`. It would be helpful if 3.2 [basic.def.odr] paragraph 6 could be amended to “look through” a `constexpr` reference in determining whether an inline function violates the ODR or not.

Proposed resolution (January, 2016):

Change 3.2 [basic.def.odr] bullet 6.2 as follows, dividing the running text into a bulleted list:

- ...Given such an entity named `D` defined in more than one translation unit, then
- each definition of `D` shall consist of the same sequence of tokens; and
 - in each definition of `D`, corresponding names, looked up according to 3.4 [basic.lookup], shall refer to an entity defined within the definition of `D`, or shall refer to the same entity, after overload resolution (13.3 [over.match]) and after matching of partial template specialization (14.8.3 [temp.over]), except that a name can refer to
 - a non-volatile `const` object with internal or no linkage if the object
 - has the same literal type in all definitions of `D`, **and**
 - **the object** is initialized with a constant expression (5.20 [expr.const]), **and**
 - **the object** is not odr-used, and
 - **the object** has the same value in all definitions of `D`,
 - or**
 - **a reference with internal or no linkage initialized with a constant expression such that the reference refers to the same entity in all definitions of `D`;**
- and
- in each definition of `D`, corresponding entities...

2106. Unclear restrictions on use of function-type template arguments

Section: 14.3.1 [temp.arg.type] **Status:** tentatively ready **Submitter:** David Krauss **Date:** 2015-03-17

According to 14.3.1 [temp.arg.type] paragraph 3,

If a declaration acquires a function type through a type dependent on a *template-parameter* and this causes a declaration that does not use the syntactic form of a function declarator to have function type, the program is ill-formed.

This is not clear enough regarding which declarations are in view. For example, does it apply to a `typedef` declaration? Does it apply to a parameter declaration, where normal function-to-pointer decay would apply? There is implementation variance at block scope.

Also, since this applies a restriction to the usage of dependent types, not template type arguments per se, the paragraph presumably should appear in 14.6.2.1 [temp.dep.type] and not its current location.

Proposed resolution (September, 2015):

1. Delete 14.3.1 [temp.arg.type] paragraph 3:

~~If a declaration acquires a function type through a type dependent on a *template-parameter* and this causes a declaration that does not use the syntactic form of a function declarator to have function type, the program is ill-formed. [Example:~~

```

template<class T> struct A {
    static T t;
};
typedef int function();


```

```
A<function> a; // ill-formed: would declare A<function>::t
// as a static member function
```

—end example]

2. Add the following as a new paragraph following 14.7 [temp.spec] paragraph 6:

...X<int> has a static member s of type int and X<char*> has a static member s of type char*. —end example]

If a function declaration acquired its function type through a dependent type (14.6.2.1 [temp.dep.type]) without using the syntactic form of a function declaator, the program is ill-formed. [Example:

```
template<class T> struct A {
    static T t;
};
typedef int function();
A<function> a; // ill-formed: would declare A<function>::t
// as a static member function
```

—end example]

2107. Lifetime of temporaries for default arguments in array copying

Section: 12.2 [class.temporary] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-03-19

The lifetime of temporaries introduced for default arguments in array copying is not specified clearly. Presumably it should be treated like default arguments in default constructors (12.2 [class.temporary] paragraph 4), which deletes each element's set of default argument temporaries before construction of the next element.

Proposed resolution (September, 2015):

Change 12.2 [class.temporary] paragraphs 4-5 as follows:

There are **two three** contexts in which temporaries are destroyed at a different point than the end of the full-expression. The first context is when a default constructor is called to initialize an element of an array **with no corresponding initializer (8.5 [dcl.init]). The second context is when a copy constructor is called to copy an element of an array while the entire array is copied (5.1.2 [expr.prim.lambda], 12.8 [class.copy]). If In either case, if** the constructor has one or more default arguments, the destruction of every temporary created in a default argument is sequenced before the construction of the next array element, if any.

The **second third** context is when a reference is bound to a temporary.¹¹⁷ The temporary to which the reference is bound...

2109. Value dependence underspecified

Section: 14.6.2.3 [temp.dep.constexpr] **Status:** tentatively ready **Submitter:** Maxim Kartashev **Date:** 2015-03-26

In the following example,

```
struct A {};

struct X {
    template <typename Q>
    int memfunc();
};

template <int (X::* P) ()> int foo(...);

template<class T> struct B {
    static int bar() {
        A a;
        return foo<&X::memfunc<T> >(a);
    }
};
```

```
template <int (X::* P) ()> int foo(A a) { return 0; }

int main() {
    return B<int>::bar();
}
```

the call `foo<&X::memfunc<T> >(a);` is dependent only if the template argument is dependent, which is only true because of the use of the template parameter `T`. Implementations generally agree that this is dependent, but there does not appear to be wording to support this determination.

Proposed resolution (September, 2015):

Change 14.6.2.3 [temp.dep.constexpr] paragraph 2 as follows:

An *id-expression* is value-dependent if:

- it is a name declared with a dependent type type-dependent,
- it is the name of a non-type template parameter,
- it names a member of an unknown specialization,
- ...

2113. Incomplete specification of types for declarators

Section: 8.3 [dcl.meaning] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-04-08

According to 8.3 [dcl.meaning]

A `static`, `thread_local`, `extern`, `register`, `mutable`, `friend`, `inline`, `virtual`, or `typedef` specifier applies directly to each *declarator-id* in an *init-declarator-list*; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

This list is missing `constexpr` and `explicit`. Also, this should apply, but doesn't, to *member-declarator-lists*.

Proposed resolution (September, 2015):

Change 8.3 [dcl.meaning] paragraph 2 as follows:

A `static`, `thread_local`, `extern`, `register`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, `explicit`, or `typedef` specifier applies directly to each *declarator-id* in an *init-declarator-list* or *member-declarator-list*; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

2122. Glvalues of void type

Section: 3.10 [basic.lval] **Status:** tentatively ready **Submitter:** CWG **Date:** 2015-05-05

According to 3.10 [basic.lval] paragraph 4,

Unless otherwise indicated (5.2.2 [expr.call]), prvalues shall always have complete types or the void type; in addition to these types, glvalues can also have incomplete types.

This wording inadvertently implies that glvalues can have type `void`, which is not correct.

Proposed resolution (January, 2016):

Change 3.10 [basic.lval] paragraph 4 as follows:

Unless otherwise indicated (5.2.2 [expr.call]), prvalues a prvalue shall always have complete types type or the void type; in addition to these types, glvalues can also have incomplete types. A glvalue shall not have type `cv void`. [Note: class A glvalue may have complete or incomplete non-void type. Class and array prvalues can have cv-qualified

types; other prvalues always have cv-unqualified types. See Clause 5 [expr]. —end note]

2129. Non-object prvalues and constant expressions

Section: 5.20 [expr.const] **Status:** tentatively ready **Submitter:** Faisal Vali **Date:** 2015-05-20

According to 5.20 [expr.const] paragraph 5,

A constant expression is either a glvalue core constant expression whose value... or a prvalue core constant expression whose value is an object where...

Since an integer literal is prvalue that is not an object, this definition does not allow it to be a constant expression.

Proposed resolution (February, 2016):

Change 5.20 [expr.const] paragraph 5 as follows:

A *constant expression* is either a glvalue core constant expression whose value refers to an entity that is a permitted result of a constant expression (as defined below), or a prvalue core constant expression whose value ~~is an object where, for that object and its subobjects~~ **satisfies the following constraints:**

- **if the value is an object of class type**, each non-static data member of reference type refers to an entity that is a permitted result of a constant expression, ~~and~~
- if the ~~object or subobject~~ **value** is of pointer type, it contains the address of an object with static storage duration, the address past the end of such an object (5.7 [expr.add]), the address of a function, or a null pointer value, ~~and~~
- **if the value is an object of class or array type, each subobject satisfies these constraints for the value.**

An entity is a *permitted result of a constant expression* if...

2140. Lvalue-to-rvalue conversion of `std::nullptr_t`

Section: 4.1 [conv.lval] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-06-12

The current rules in 4.1 [conv.lval] paragraph 2 do not require fetching the value in memory of an object of type `std::nullptr_t` in order to produce its prvalue. This choice has implications that may not have been considered for questions like whether use of a `std::nullptr_t` that is an inactive member of a union results in undefined behavior or whether a volatile `std::nullptr_t` variable in a discarded-value expression produces a side effect.

Proposed resolution (January, 2016):

Change 4.1 [conv.lval] bullet 2.3 as follows:

...In all other cases, the result of the conversion is determined according to the following rules:

- If τ is (possibly cv-qualified) `std::nullptr_t`, the result is a null pointer constant (4.10 [conv.ptr]). **[Note: Since no value is fetched from memory, there is no side effect for a volatile access (1.9 [intro.execution]), and an inactive member of a union (9.5 [class.union]) may be accessed. —end note]**
 - ...
-

2141. Ambiguity in *new-expression* with *elaborated-type-specifier*

Section: 5.3.4 [expr.new] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-06-12

Consider the following example:

```
struct A { };

void foo() {
    new struct A { };
}
```

This could be either an *elaborated-type-specifier* followed by a *braced-init-list* or a *class-specifier*. There does not appear to be a disambiguation rule for this case.

One possibility for addressing this could be to use *trailing-type-specifier-seq* instead of *type-specifier-seq* in *new-type-id*. That could also be a purely syntactic alternative to the resolution of issue 686: change all uses of *type-specifier-seq* to *trailing-type-specifier-seq* and provide a new grammar production for use in *alias-declaration*.

Proposed resolution (February, 2016):

1. Change the grammar in 7 [dcl.dcl] paragraph 1 as follows:

alias-declaration:
 using *identifier* *attribute-specifier-seq*_{opt} = **defining**-*type-id*

2. Change 7 [dcl.dcl] paragraph 8 as follows:

Each *init-declarator* in the *init-declarator-list* contains exactly one *declarator-id*, which is the name declared by that *init-declarator* and hence one of the names declared by the declaration. The **defining**-*type-specifiers* (7.1.6 [dcl.type]) in the *decl-specifier-seq* and the recursive declarator structure of the *init-declarator* describe a type (8.3 [dcl.meaning]), which is then associated with the name being declared by the *init-declarator*.

3. Change the grammar in 7.1 [dcl.spec] paragraph 1 as follows:

decl-specifier:
storage-class-specifier
defining-*type-specifier*
function-specifier
friend
typedef
constexpr

4. Change 7.1 [dcl.spec] paragraph 3 as follows:

If a *type-name* is encountered while parsing a *decl-specifier-seq*, it is interpreted as part of the *decl-specifier-seq* if and only if there is no previous **defining**-*type-specifier* other than a *cv-qualifier* in the *decl-specifier-seq*. The sequence...

5. Change 7.1.3 [dcl.typedef] paragraph 1 as follows:

Declarations containing the *decl-specifier* *typedef* declare identifiers that can be used later for naming fundamental (3.9.1 [basic.fundamental]) or compound (3.9.2 [basic.compound]) types. The *typedef* specifier shall not be combined in a *decl-specifier-seq* with any other kind of specifier except a **defining**-*type-specifier*, and it shall not be used in the *decl-specifier-seq* of a *parameter-declaration* (8.3.5 [dcl.fct]) nor in the *decl-specifier-seq* of a *function-definition* (8.4 [dcl.fct.def]).

6. Change 7.1.3 [dcl.typedef] paragraph 2 as follows:

A *typedef-name* can also be introduced by an *alias-declaration*. The identifier following the *using* keyword becomes a *typedef-name* and the optional *attribute-specifier-seq* following the identifier appertains to that *typedef-name*. **It The defining-type-specifier-seq of the defining-type-id may define a class or enumeration only if the alias-declaration is not the declaration of a template-declaration. Such a typedef-name** has the same semantics as if it were introduced by the *typedef* specifier. In particular, it does not define a new type.
 [Example:

7. Change 7.1.6 [dcl.type] paragraph 1 as follows:

type-specifier:
~~*trailing-type-specifier*~~
~~*class-specifier*~~
~~*enum-specifier*~~
~~*trailing-type-specifier*~~:
simple-type-specifier

~~elaborated-type-specifier~~
~~typename-specifier~~
~~cv-qualifier~~
type-specifier-seq:
type-specifier attribute-specifier-seq_{opt}
type-specifier type-specifier-seq
defining-type-specifier:
type-specifier
class-specifier
enum-specifier
~~trailing~~**defining**-type-specifier-seq:
~~trailing~~**defining**-type-specifier attribute-specifier-seq_{opt}
~~trailing~~**defining**-type-specifier ~~trailing~~**defining**-type-specifier-seq

The optional *attribute-specifier-seq* in a *type-specifier-seq* or a ~~trailing~~**defining**-*type-specifier-seq* appertains to the type denoted by the preceding *type-specifiers* **or defining-type-specifiers** (8.3 [dcl.meaning]). The *attribute-specifier-seq* affects the type only for the declaration it appears in, not other declarations involving the same type.

8. Change 7.1.6.2 [dcl.type.simple] paragraph 2 as follows:

As a general rule, at most one **defining**-*type-specifier* is allowed in the complete *decl-specifier-seq* of a declaration or in a *type-specifier-seq* or ~~trailing~~**defining**-*type-specifier-seq*. The only exceptions to this rule are the following:...

9. Change 7.1.6 [dcl.type] paragraph 3 as follows:

Except in a declaration of a constructor, destructor, or conversion function, at least one **defining**-*type-specifier* that is not a *cv-qualifier* shall appear in a complete *type-specifier-seq* or a complete *decl-specifier-seq*.⁹⁵ ~~A type-specifier-seq shall not define a class or enumeration unless it appears in the type-id of an alias-declaration (7.1.3 [dcl.typedef]) that is not the declaration of a template-declaration.~~

10. Change the grammar in 8 [dcl.decl] paragraph 4 as follows:

trailing-return-type:
-> ~~trailing-type-specifier-seq abstract-declarator_{opt}~~ **type-id**

11. Change the grammar in 8.1 [dcl.name] paragraph 1 as follows:

type-id:
type-specifier-seq abstract-declarator_{opt}
defining-type-id:
defining-type-specifier-seq abstract-declarator_{opt}

12. Change 12.3.2 [class.conv.fct] paragraph 1 as follows:

...A *decl-specifier* in the *decl-specifier-seq* of a conversion function (if any) shall be neither a **defining**-*type-specifier* nor static. Type of the conversion function

2143. Value-dependency via injected-class-name

Section: 14.6.2.1 [temp.dep.type] **Status:** tentatively ready **Submitter:** Maxim Kartashev **Date:** 2015-06-16

There does not appear to be a rule that causes `p` or `this` to be dependent in the following example:

```

template <typename T> struct A {
    void foo() {
        A* p = 0;
        bar(p);    // will be found by ADL at the point of instantiation
        bar(this); // same here
    }
};

void bar(...);

```

```
int main() {
    A<int> a;
    a.foo();
}
```

Proposed resolution (February, 2016):

Change 14.6.2.1 [temp.dep.type] bullet 9.7 as follows:

A type is dependent if it is

- ...
- **an injected-type-name (Clause 9 [class] of a class template or** a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent or is a pack expansion, or
- denoted by...

2146. Scalar object vs memory location in definition of “unsequenced”

Section: 1.9 [intro.execution] **Status:** tentatively ready **Submitter:** Jens Maurer **Date:** 2015-06-22

According to 1.9 [intro.execution] paragraph 15,

If a side effect on a scalar object is unsequenced relative to either another side effect on the same scalar object or a value computation using the value of the same scalar object, and they are not potentially concurrent (1.10 [intro.multithread]), the behavior is undefined.

Should this refer to “memory location,” which also encompasses contiguous bit-fields, as the definition of data races in 1.10 [intro.multithread] does? For example,

```
struct S {
    int x : 4;
    int y : 4;
    int z : 4;
};

void f(int, int, int);
int g(int, S&);

int main(int argc, char ** argv) {
    S s = { argc, argc+1, argc+2 };
    f(++s.x, g(++s.y, s), ++s.z);
}
```

Proposed resolution (February, 2016):

Change 1.9 [intro.execution] paragraph 15 as follows:

...If a side effect on a **scalar object memory location (1.7 [intro.memory])** is unsequenced relative to either another side effect on the same **scalar object memory location** or a value computation using the value of **any object in** the same **scalar object memory location**, and they are not potentially concurrent (1.10 [intro.multithread]), the behavior is undefined. [Note: The next section...

2147. Initializer-list arguments and pack deduction

Section: 14.8.2.1 [temp.deduct.call] **Status:** tentatively ready **Submitter:** Hubert Tong **Date:** 2015-06-22

The current wording of 14.8.2.1 [temp.deduct.call] paragraph 1 dealing with deduction for a trailing parameter pack refers to “the type” of the argument, which does not apply to an initializer list. There is implementation divergence in the handling of an example like

```
#include <initializer_list>

template <typename... T>
void q(std::initializer_list<std::initializer_list<T>>... tt);

void bar() { q({{0}}, {{'\0'}}); }
```

Proposed resolution (September, 2015):

Change 14.8.2.1 [temp.deduct.call] paragraph 1 as follows:

...For a function parameter pack that occurs at the end of the *parameter-declaration-list*, the type *A* of **deduction is performed for** each remaining argument of the call, **is compared with taking** the type *P* of the *declarator-id* of the function parameter pack **as the corresponding function template parameter type**. Each **comparison deduction** deduces template arguments for subsequent positions in the template parameter packs expanded by the function parameter pack. When a function parameter pack appears in a non-deduced context (14.8.2.5 [temp.deduct.type]), the type of that parameter pack is never deduced. [Example:...

2153. *pure-specifier* in friend declaration

Section: 9.2 [class.mem] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-06-30

The current wording does not appear to ban a *pure-specifier* from a friend declaration.

Proposed resolution (February, 2016):

Change 9.2 [class.mem] paragraph 6 as follows:

...A *pure-specifier* shall be used only in the declaration of a virtual function (10.3 [class.virtual]) **that is not a friend declaration**.

2154. Ambiguity of *pure-specifier*

Section: 9.2 [class.mem] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-06-30

There does not appear to be a rule to disambiguate a *pure-specifier* and a *brace-or-equal-initializer* in a member declarator.

Proposed resolution (February, 2016):

Add the following as a new paragraph following 9.2 [class.mem] paragraph 3:

[Note: A single name can denote several function members provided their types are sufficiently different (Clause 13 [over])). —end note]

In a *member-declarator*, an = immediately following the *declarator* is interpreted as introducing a *pure-specifier* if the *declarator-id* has function type, otherwise it is interpreted as introducing a *brace-or-equal-initializer*. [Example:

```
struct S {
    using T = void();
    T * p = 0;           // OK: brace-or-equal-initializer
    virtual T f = 0;     // OK: pure-specifier
};
```

—end example]

2156. Definition of enumeration declared by *using-declaration*

Section: 7.2 [dcl.enum] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-07-06

The description of enumeration declarations in 7.2 [dcl.enum] does not, but should, contain similar wording to that preventing a class definition from defining a class type named by a *using-declaration*:

If a *class-head-name* contains a *nested-name-specifier*, the *class-specifier* shall refer to a class that was previously declared directly in the class or namespace to which the *nested-name-specifier* refers, or in an element of the inline namespace set (7.3.1 [namespace.def]) of that namespace (i.e., not merely inherited or introduced by a *using-declaration*), and the *class-specifier* shall appear in a namespace enclosing the previous declaration. In such cases, the *nested-name-specifier* of the *class-head-name* of the definition shall not begin with a *decltype-specifier*.

Proposed resolution (February, 2016):

Add the following as a new paragraph at the end of 7.2 [dcl.enum]:

If an *enum-head* contains a *nested-name-specifier*, the *enum-specifier* shall refer to an enumeration that was previously declared directly in the class or namespace to which the *nested-name-specifier* refers, or in an element of the inline namespace set (7.3.1 [namespace.def]) of that namespace (i.e., not merely inherited or introduced by a *using-declaration*), and the *enum-specifier* shall appear in a namespace enclosing the previous declaration. In such cases, the *nested-name-specifier* of the *enum-head* of the definition shall not begin with a *decltype-specifier*.

2162. Capturing this by reference

Section: 5.1.2 [expr.prim.lambda] **Status:** tentatively ready **Submitter:** Brian Bi **Date:** 2015-07-23

The current wording of 5.1.2 [expr.prim.lambda] paragraphs 15 and 16 does not, but presumably should, indicate that *this*, if implicitly captured, is always captured by copy and not by reference.

Proposed resolution (February, 2016):

Change 5.1.2 [expr.prim.lambda] paragraph 15 as follows, dividing the existing text into a bulleted list:

An entity is *captured by copy* if

- it is *this*,
- it is implicitly captured and the *capture-default* is `=`, or
- if it is explicitly captured with a capture that is not of the form `& identifier` or `& identifier initializer`.

For each entity captured by copy...

2163. Labels in constexpr functions

Section: 7.1.5 [dcl.constexpr] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-07-24

The requirements for `constexpr` functions do not, but presumably should, forbid the appearance of a label in the function body (gotos are prohibited).

Proposed resolution (January, 2016):

Add the following as an additional bullet following 7.1.5 [dcl.constexpr] bullet 3.5.2:

The definition of a `constexpr` function shall satisfy the following requirements:

- ...
- its *function-body* shall be `= delete`, `= default`, or a *compound-statement* that does not contain
 - an *asm-definition*,
 - a `goto` statement,

- **an identifier label (6.1 [stmt.label]),**
- ...

2167. Non-member references with lifetimes within the current evaluation

Section: 5.20 [expr.const] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-08-11

The current wording of 5.20 [expr.const] bullet 2.9 says:

- an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
 - it is initialized with a constant expression or
 - it is a non-static data member of an object whose lifetime began within the evaluation of *e*;

This incorrectly excludes non-member references whose lifetime began within the current evaluation.

Proposed resolution (February, 2016):

Change 5.20 [expr.const] bullet 2.9.2 as follows:

A *conditional-expression* *e* is a *core constant expression* unless the evaluation of *e*, following the rules of the abstract machine (1.9 [intro.execution]), would evaluate one of the following expressions:

- ...
- an *id-expression* that refers to a variable or data member of reference type unless the reference has a preceding initialization and either
 - it is initialized with a constant expression or
 - ~~it is a non-static data member of an object whose~~ **its** lifetime began within the evaluation of *e*;
- ...

2175. Ambiguity with attribute in conversion operator declaration

Section: 8.2 [dcl.ambig.res] **Status:** tentatively ready **Submitter:** Daveed Vandevorde **Date:** 2015-09-17

The declaration

```
operator int [[noreturn]] ();
```

is ambiguous with respect to the binding of the attribute. It could either be parsed (as apparently intended by the user) as part of the *noctr-declarator* (8 [dcl.decl] paragraph 4) or as part of the *type-specifier-seq* (7.1.6 [dcl.type] paragraph 1) of the *conversion-type-id* (12.3.2 [class.conv.fct] paragraph 1). Current implementations disambiguate this declaration in favor of the latter interpretation, issuing an error for the declaration because the `noreturn` attribute cannot apply to a type.

Proposed resolution (February, 2016):

Change 12.3.2 [class.conv.fct] paragraph 3 as follows:

The *conversion-type-id* shall not represent a function type nor an array type. The *conversion-type-id* in a *conversion-function-id* is the longest ~~possible~~ sequence of ~~conversion-declarators~~ **tokens that could possibly form a conversion-type-id**. [Note: This prevents ambiguities between the declarator operator `*` and its expression counterparts. [Example:

```
&ac.operator int*i; // syntax error:
// parsed as: &(ac.operator int *)i
// not as: &(ac.operator int)*i
```

The * is the pointer declarator and not the multiplication operator. —end example] **This rule also prevents ambiguities for attributes. [Example:**

```
operator int [[noreturn]] (); // error: noreturn attribute applied to a type
```

—end example] —end note]

2176. Destroying the returned object when a destructor throws

Section: 5.2.2 [expr.call] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-09-28

Consider the following example:

```
#include <stdio.h>

struct X {
    X() { puts("X()"); }
    X(const X&) { puts("X(const X&)"); }
    ~X() { puts("~X()"); }
};

struct Y { ~Y() noexcept(false) { throw 0; } };

X f() {
    try {
        Y y;
        return {};
    } catch (...) {
    }
    return {};
}

int main() {
    f();
}
```

Current implementations print `x()` twice but `~x()` only once. That is obviously wrong, but it is not clear that the current wording covers this case.

Proposed resolution (February, 2016):

Change 15.2 [except.ctor] paragraph 2 as follows:

The destructor is invoked for each automatic object of class type constructed, **but not yet destroyed**, since the try block was entered. **If an exception is thrown during the destruction of temporaries or local variables for a return statement (6.6.3 [stmt.return]), the destructor for the returned object (if any) is also invoked.** The **automatic** objects are destroyed in the reverse order of the completion of their construction. **[Example:**

```
struct A { };
```

```
struct Y { ~Y() noexcept(false) { throw 0; } };
```

```
A f() {
    try {
        A a;
        Y y;
        A b;

        return {}; // #1
    } catch (...) {
    }

    return {}; // #2
}
```

At #1, the returned object of type A is constructed. Then, the local variable b is destroyed (6.6 [stmt.jump]). Next, the local variable y is destroyed, causing stack unwinding, resulting in the destruction of the returned object, followed by the destruction of the local variable a. Finally, the returned object is constructed again at #2. —end example]

2180. Virtual bases in destructors and defaulted assignment operators

Section: 12.8 [class.copy] **Status:** tentatively ready **Submitter:** Vinny Romano **Date:** 2015-10-13

According to 12.8 [class.copy] paragraph 28,

The implicitly-defined copy/move assignment operator for a non-union class X performs memberwise copy/move assignment of its subobjects. The direct base classes of X are assigned first, in the order of their declaration in the *base-specifier-list*, and then the immediate non-static data members of X are assigned, in the order in which they were declared in the class definition... It is unspecified whether subobjects representing virtual base classes are assigned more than once by the implicitly-defined copy/move assignment operator.

However, the determination of whether a defaulted copy/move assignment operator is defined as deleted (paragraph 23) considers only the “potentially constructed” subobjects:

A defaulted copy/move assignment operator for class x is defined as deleted if x has:

- ...
- a potentially constructed subobject of class type M (or array thereof) that cannot be copied/moved...

where “potentially constructed” is defined (in 12 [special] paragraph 5) as:

For a class, its non-static data members, its non-virtual direct base classes, and, if the class is not abstract (10.4 [class.abstract]), its virtual base classes are called its *potentially constructed subobjects*.

i.e., excluding direct virtual base classes of abstract classes. This seems contradictory, since an implementation is expressly permitted to assign such base classes and thus presumably is permitted to fail if no such assignment is possible.

Similarly, 12.4 [class.dtor] paragraph 8 says,

After executing the body of the destructor and destroying any automatic objects allocated within the body, a destructor for class x calls the destructors for x's direct non-variant non-static data members, the destructors for x's direct base classes and, if x is the type of the most derived class (12.6.2 [class.base.init]), its destructor calls the destructors for x's virtual base classes.

This appears to allow a virtual base's destructor to be called more than once, once for each class naming it as a direct virtual base and once for the most-derived class.

Proposed resolution (February, 2016):

1. Change 12.4 [class.dtor] paragraph 8 as follows:

After executing the body of the destructor and destroying any automatic objects allocated within the body, a destructor for class x calls the destructors for x's direct non-variant non-static data members, the destructors for x's **non-virtual** direct base classes and, if x is the type of the most derived class (12.6.2 [class.base.init]), its destructor calls the destructors for x's virtual base classes. All destructors are called as if...

2. Change 12.8 [class.copy] bullet 23.4 as follows:

A defaulted copy/move assignment operator for class x is defined as deleted if x has:

- ...
- ~~a potentially constructed subobject~~ **direct non-static data member** of class type M (or array thereof) **or a direct base class M** that cannot be copied/moved because overload resolution (13.3 [over.match]), as applied to M's corresponding assignment operator, results in an ambiguity or a function that is deleted or inaccessible from the defaulted assignment operator.